

COMPLETE ROADMAP & COURSEWORK

Go

Simple, fast, concurrent — built for the modern cloud

Why Google built Go, the type system, goroutines, channels, the standard library, and the full ecosystem

[7 Phases](#) · [Roadmap](#) · [Full Coursework with Code](#) · [Complete Reference](#)

7

PHASES

50+

CONCEPTS

20

PROJECTS

~5mo

TIMELINE

1

CONCURRENCY
MODEL TO MASTER

PHASE 0 Why Go Exists

The engineering problems at Google that created Go

0.1 The Origin Story

In 2007, three engineers at Google — Robert Griesemer, Rob Pike, and Ken Thompson — were waiting for a C++ codebase to compile. They decided to sketch a new language on a whiteboard. Go 1.0 shipped in 2012.

The frustrations they were solving were real and specific: C++ compilation took hours on large codebases, Java had too much ceremony for server code, Python was too slow for systems work, and no existing language made concurrent network servers easy to write correctly.

Problem	Go's Answer
C++ compile times measured in hours	Go compiles a large project in seconds — the compiler is a first-class feature
Concurrency is hard (threads, locks, deadlocks)	Goroutines + channels: CSP-based concurrency that is safe by design
Dependency management was a mess	Built-in module system with go.mod — reproducible builds by default
Too many ways to do the same thing	Opinionated: one way to format (gofmt), one error handling style, minimal syntax
Large teams can't agree on style	gofmt enforces a single style — no style debates, ever
GC pauses were too long for server code	Sub-millisecond GC pauses — suitable for latency-sensitive services

Go's Philosophy

Do less, enable more. Go deliberately omits features: no classes (use structs + methods), no inheritance (use composition), no exceptions (errors are values), no generics until 1.18. Every omission was intentional — complexity is the enemy of correctness.

0.2 The Roadmap at a Glance

Phase 1 — Go Fundamentals

Phase 2 — Composite Types

Variables, types, functions, control flow, packages, and the Go toolchain. Phase 3 — Error Handling Errors as values, custom errors, wrapping, sentinel errors, panic/recover.	Arrays, slices, maps, structs, methods, and interfaces. Phase 4 — Concurrency Goroutines, channels, select, sync primitives, context, patterns.
Phase 5 — Standard Library io, net/http, encoding/json, os, strings, time, context.	Phase 6 — Modules & Tooling go mod, testing, benchmarks, race detector, profiling, linting.
Phase 7 — Advanced Go Generics, reflection, cgo, build tags, popular frameworks and patterns.	

PHASE 1 Go Fundamentals

Variables, types, functions, and how Go thinks

1.1 Variables and Types

Go is statically typed with type inference. Every variable has a type known at compile time. Go's type system is strict — there are no implicit conversions.

```
package main

import "fmt"

func main() {
    // var declaration — explicit type
    var name string = "Alice"
    var age  int    = 30
    var pi   float64 = 3.14159

    // Short variable declaration — type inferred (most common inside functions)
    city    := "Nairobi"
    count   := 0
    active  := true

    // Multiple assignment
    x, y := 10, 20
    x, y = y, x // swap — no temp variable needed

    // Zero values — Go initialises everything to its zero value
    var i int      // 0
    var s string   // "" (empty string)
    var b bool     // false
    var p *int     // nil

    // Constants
    const MaxRetries = 3
    const Pi = 3.14159265358979

    // iota — auto-incrementing constants
    const (
        Sunday = iota // 0
        Monday  // 1
        Tuesday  // 2
    )

    fmt.Println(name, age, pi, city, count, active, x, y, i, s, b, p)
}
```

Numeric Types

Type	Size	Range / Notes
int	Platform-size (64-bit on 64-bit OS)	Use for general integers — most common
int8 / int16 / int32 / int64	8/16/32/64 bits	Use when size matters (protocols, files)
uint / uint8 / uint16 / uint32 / uint64	unsigned	byte = uint8, rune = int32 (Unicode code point)
float32 / float64	32 / 64 bit	float64 is the default — use for most math
complex64 / complex128	complex numbers	Rare — for scientific computing
string	immutable UTF-8 bytes	Not a byte array — use []byte for mutation

1.2 Functions

```
package main

// Basic function
func add(a, b int) int {
    return a + b
}

// Multiple return values - idiomatic Go
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}

// Named return values
func minMax(arr []int) (min, max int) {
    min, max = arr[0], arr[0]
    for _, v := range arr {
        if v < min { min = v }
        if v > max { max = v }
    }
    return // naked return - returns named values
}

// Variadic function - accepts any number of args
func sum(nums ...int) int {
    total := 0
    for _, n := range nums { total += n }
    return total
}

sum(1, 2, 3)           // 6
sum([]int{1,2,3}...)  // spread a slice
```

```
// First-class functions
func apply(f func(int) int, x int) int { return f(x) }
double := func(x int) int { return x * 2 }
apply(double, 5) // 10

// Defer – runs when the enclosing function returns (LIFO order)
func readFile(name string) error {
    f, err := os.Open(name)
    if err != nil { return err }
    defer f.Close() // guaranteed to run even if we panic or return early
    // ... read from f
    return nil
}
```

1.3 Control Flow

```
// if – no parentheses, but braces are required
if x > 0 {
    fmt.Println("positive")
} else if x < 0 {
    fmt.Println("negative")
} else {
    fmt.Println("zero")
}

// if with initialisation statement – very idiomatic
if err := doSomething(); err != nil {
    return fmt.Errorf("failed: %w", err)
}

// for – Go's only loop (replaces while, do-while, for)
for i := 0; i < 10; i++ { } // traditional
for i < 10 { i++ }         // while-style
for { break }              // infinite loop
for i, v := range slice { } // range over slice
for k, v := range myMap { } // range over map
for i, r := range "hello" { } // range over string (runes)
for v := range ch { }       // range over channel

// switch – no fallthrough by default
switch day {
case "Mon", "Tue", "Wed", "Thu", "Fri":
    fmt.Println("Weekday")
case "Sat", "Sun":
    fmt.Println("Weekend")
default:
    fmt.Println("Unknown")
}

// Type switch – check dynamic type of an interface
switch v := i.(type) {
case int:    fmt.Printf("int: %d\n", v)
case string: fmt.Printf("string: %s\n", v)
default:    fmt.Printf("unknown: %T\n", v)}
```

```
}
```

Phase 1 Projects



CLI Calculator

A REPL calculator that reads expressions from stdin, evaluates them, and prints results. Covers functions, error handling basics, fmt, bufio, and os.

Concepts: [functions](#) · [multiple returns](#) · [fmt](#) · [bufio](#) · [os.Stdin](#)



File Stats Tool

A CLI tool that reads a text file and outputs: line count, word count, character count, most frequent word. Uses os, bufio, strings, and maps.

Concepts: [maps](#) · [slices](#) · [strings](#) · [os](#) · [bufio](#) · [range](#)

PHASE 2 Composite Types

Slices, maps, structs, methods, and interfaces

2.1 Slices — Go's Most Important Data Structure

A slice is a view into an underlying array. It has a pointer, a length, and a capacity. Understanding this model prevents many common bugs.

```
// Creating slices
s1 := []int{1, 2, 3}           // literal
s2 := make([]int, 5)          // len=5, cap=5, all zeros
s3 := make([]int, 3, 10)       // len=3, cap=10

// Appending — may allocate a new underlying array if cap exceeded
s1 = append(s1, 4, 5)          // [1 2 3 4 5]
s1 = append(s1, s2...)         // append another slice

// Slicing — creates a new slice header sharing the underlying array
a := []int{0, 1, 2, 3, 4, 5}
b := a[1:4] // [1 2 3] — shares memory with a
b[0] = 99    // ALSO changes a[1] to 99!

// Copy — creates an independent slice
dst := make([]int, len(a))
n := copy(dst, a) // copies min(len(dst), len(a)) elements

// 2D slice
matrix := make([][]int, 3)
for i := range matrix { matrix[i] = make([]int, 3) }

// Common patterns
// Delete at index i (order preserved):
s = append(s[:i], s[i+1:]...)
// Delete at index i (order not preserved — fast):
s[i] = s[len(s)-1]; s = s[:len(s)-1]
// Filter in place:
n := 0
for _, v := range s {
    if keep(v) { s[n] = v; n++ }
}
s = s[:n]
```

2.2 Maps

```
// Creating maps
```

```
m1 := map[string]int{}
m2 := map[string]int{"alice": 1, "bob": 2}
m3 := make(map[string]int) // prefer make for clarity

// Operations
m3["key"] = 42 // insert or update
val := m3["key"] // 42
val, ok := m3["missing"] // ok=false, val=0 (zero value) – ALWAYS use the two-
value form
delete(m3, "key")
length := len(m3)

// Iterating – order is intentionally random in Go
for k, v := range m3 {
    fmt.Printf("%s: %d\n", k, v)
}

// For deterministic order, sort the keys first:
keys := make([]string, 0, len(m3))
for k := range m3 { keys = append(keys, k) }
sort.Strings(keys)
for _, k := range keys { fmt.Println(k, m3[k]) }

// Map of slices – common pattern
groups := make(map[string][]string)
groups["fruits"] = append(groups["fruits"], "apple", "banana")

// Checking if a key exists before use
if count, ok := wordFreq[word]; ok {
    wordFreq[word] = count + 1
} else {
    wordFreq[word] = 1
}
// Equivalent idiomatic form:
wordFreq[word]++ // zero value of int is 0, so this works even if key doesn't
exist
```

2.3 Structs and Methods

```
package main

// Struct definition
type Point struct {
    X, Y float64
}

type Rectangle struct {
    TopLeft    Point
    BottomRight Point
}

// Method on a value receiver – operates on a copy
func (p Point) Distance(q Point) float64 {
    dx := p.X - q.X
```

```

    dy := p.Y - q.Y
    return math.Sqrt(dx*dx + dy*dy)
}

// Method on a pointer receiver — can modify the struct
// Rule: if any method needs a pointer receiver, give ALL methods pointer
receivers
func (r *Rectangle) Scale(factor float64) {
    r.BottomRight.X = r.TopLeft.X + (r.BottomRight.X-r.TopLeft.X)*factor
    r.BottomRight.Y = r.TopLeft.Y + (r.BottomRight.Y-r.TopLeft.Y)*factor
}

func (r Rectangle) Area() float64 {
    w := r.BottomRight.X - r.TopLeft.X
    h := r.BottomRight.Y - r.TopLeft.Y
    return math.Abs(w * h)
}

// Struct embedding — Go's composition mechanism
type Animal struct { Name string }
func (a Animal) Speak() string { return a.Name + " makes a sound" }

type Dog struct {
    Animal          // embed — Dog inherits Animal's fields and methods
    Breed string
}
func (d Dog) Speak() string { return d.Name + " barks" } // override

d := Dog{Animal: Animal{Name: "Rex"}, Breed: "Labrador"}
d.Name // Rex (promoted from Animal)
d.Speak() // 'Rex barks' (Dog's method shadows Animal's)
d.Animal.Speak() // 'Rex makes a sound' (explicit access)

```

2.4 Interfaces

An interface in Go is a set of method signatures. A type implements an interface implicitly — no 'implements' keyword. This is structural typing, also called duck typing.

```

// Define an interface
type Shape interface {
    Area() float64
    Perimeter() float64
}

// Any type with these methods automatically implements Shape
type Circle struct { Radius float64 }
func (c Circle) Area() float64 { return math.Pi * c.Radius * c.Radius }
func (c Circle) Perimeter() float64 { return 2 * math.Pi * c.Radius }

type Rect struct { W, H float64 }
func (r Rect) Area() float64 { return r.W * r.H }
func (r Rect) Perimeter() float64 { return 2 * (r.W + r.H) }

```

```
// Use the interface polymorphically
func printShape(s Shape) {
    fmt.Printf("Area: %.2f, Perimeter: %.2f\n", s.Area(), s.Perimeter())
}
printShape(Circle{Radius: 5})
printShape(Rect{W: 3, H: 4})

// The empty interface – accepts any value (like 'any' in TS)
func printAnything(v interface{}) { fmt.Println(v) }
func printAnything(v any) { fmt.Println(v) } // 'any' is alias for interface{}

// Important built-in interfaces:
// error    – Error() string
// Stringer – String() string (used by fmt.Println)
// Reader   – Read(p []byte) (n int, err error)
// Writer   – Write(p []byte) (n int, err error)

// Implement Stringer so fmt knows how to print your type
func (p Point) String() string { return fmt.Sprintf("(%g, %g)", p.X, p.Y) }
```

Phase 2 Projects



Zoo with Interfaces

Define an Animal interface (Speak, Eat, Move). Implement Dog, Cat, Bird, Fish. Write a Zoo struct that holds []Animal and a DailyRoutine() method that calls each animal's methods. No type assertions needed.

Concepts: [interfaces](#) · [structs](#) · [methods](#) · [slices](#) · [embedding](#)

PHASE 3 Error Handling*Errors as values — Go's explicit, composable approach*

3.1 Errors as Values

Go has no exceptions. Errors are ordinary values of the built-in 'error' interface. A function that can fail returns (result, error). The caller must explicitly handle the error or pass it up. This makes error paths visible in the code.

```
// The error interface — just one method
type error interface {
    Error() string
}

// Returning errors
func openFile(name string) (*os.File, error) {
    f, err := os.Open(name)
    if err != nil {
        return nil, fmt.Errorf("openFile: %w", err) // wrap with context
    }
    return f, nil
}

// Handling errors — check immediately after the call
f, err := openFile("data.txt")
if err != nil {
    log.Fatalf("Failed to open file: %v", err)
}
defer f.Close()

// Creating simple errors
errors.New("something went wrong")           // static message
fmt.Errorf("user %d not found", userID)      // formatted message
fmt.Errorf("load config: %w", err)           // %w wraps the original error
```

3.2 Custom Errors and Sentinel Errors

```
package myapp

import "errors"

// Sentinel errors — package-level error values
var ErrNotFound = errors.New("not found")
var ErrUnauthorized = errors.New("unauthorized")
```

```
// Custom error type – carries extra context
type ValidationError struct {
    Field   string
    Message string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("validation failed: %s – %s", e.Field, e.Message)
}

// Returning typed errors
func validateEmail(email string) error {
    if !strings.Contains(email, "@") {
        return &ValidationError{Field: "email", Message: "must contain @"}
    }
    return nil
}

// errors.Is – check if an error IS a specific sentinel error (unwraps chain)
if errors.Is(err, ErrNotFound) {
    // handle not found
}

// errors.As – check if an error IS a specific type and extract it
var valErr *ValidationError
if errors.As(err, &valErr) {
    fmt.Printf("Validation failed on field: %s\n", valErr.Field)
}

// errors.Unwrap – get the wrapped error
wrapped := fmt.Errorf("outer: %w", ErrNotFound)
errors.Unwrap(wrapped) // returns ErrNotFound
errors.Is(wrapped, ErrNotFound) // true – checks whole chain
```

3.3 panic and recover

```
// panic – signals a programming error (not expected errors)
// Use for: impossible states, violated invariants, nil pointer dereferences
func mustParse(s string) int {
    n, err := strconv.Atoi(s)
    if err != nil {
        panic(fmt.Sprintf("mustParse: invalid input %q", s))
    }
    return n
}

// recover – stops a panic from crashing the program
// Must be called from a deferred function
func safeDiv(a, b int) (result int, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("recovered from panic: %v", r)
        }
    }()
    return a / b, nil // panics if b == 0
}
```

```
}

// Real-world use: HTTP server middleware that recovers from panics in handlers
func panicMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                log.Printf("panic: %v\n%s", err, debug.Stack())
                http.Error(w, "Internal Server Error", 500)
            }
        }()
        next.ServeHTTP(w, r)
    })
}

// Rule: use error returns for expected failures, panic for impossible states
```

Phase 3 Projects



Bank Account with Typed Errors

A bank account package with custom error types: `InsufficientFundsError`, `NegativeAmountError`, `AccountNotFoundError`. All operations return errors. Use `errors.Is` and `errors.As` in tests.

Concepts: [custom errors](#) · [sentinel errors](#) · [errors.Is](#) · [errors.As](#) · [fmt.Errorf](#) %w

PHASE 4 Concurrency

Goroutines, channels, select, and sync primitives

4.1 Goroutines — Lightweight Threads

A goroutine is a function running concurrently with other goroutines in the same address space. It starts with 'go'. Go can run millions of goroutines simultaneously — each starts with only 2KB of stack.

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    // Start goroutines with 'go'
    go fmt.Println("Hello from goroutine 1")
    go fmt.Println("Hello from goroutine 2")
    // Main goroutine continues immediately

    // WaitGroup - wait for goroutines to complete
    var wg sync.WaitGroup

    for i := 0; i < 5; i++ {
        wg.Add(1) // increment counter
        go func(id int) { // pass id as arg to avoid closure capture bug
            defer wg.Done() // decrement counter when done
            fmt.Printf("Worker %d started\n", id)
            time.Sleep(100 * time.Millisecond)
            fmt.Printf("Worker %d done\n", id)
        }(i)
    }

    wg.Wait() // blocks until counter reaches zero
    fmt.Println("All workers done")
}

// The goroutine closure capture bug - IMPORTANT to understand:
for i := 0; i < 3; i++ {
    go func() { fmt.Println(i) }() // WRONG: all print 3 (captures loop var)
}
for i := 0; i < 3; i++ {
    go func(i int) { fmt.Println(i) }(i) // CORRECT: copy i into func arg
}
```

4.2 Channels — Communication Between Goroutines

Channels are typed pipes for goroutines to communicate. Go's motto: 'Do not communicate by sharing memory; share memory by communicating.'

```
// Unbuffered channel — sender blocks until receiver is ready
ch := make(chan int)

go func() { ch <- 42 }() // send (blocks until received)
val := <-ch             // receive (blocks until sent)
fmt.Println(val)        // 42

// Buffered channel — sender only blocks when buffer is full
buffered := make(chan string, 3) // buffer of 3
buffered <- "a" // does not block
buffered <- "b" // does not block
buffered <- "c" // does not block
// buffered <- "d" // would block — buffer full

// Directional channels — restrict channel usage
func producer(ch chan<- int) { // send-only
    for i := 0; i < 5; i++ { ch <- i }
    close(ch) // signal no more values
}
func consumer(ch <-chan int) { // receive-only
    for v := range ch { // range over channel until closed
        fmt.Println(v)
    }
}

// Pipeline pattern — chain goroutines with channels
func generate(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        for _, n := range nums { out <- n }
        close(out)
    }()
    return out
}

func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        for n := range in { out <- n * n }
        close(out)
    }()
    return out
}

for v := range square(generate(2, 3, 4)) {
    fmt.Println(v) // 4, 9, 16
}
```

4.3 select — Non-Blocking Channel Operations

```
// select waits on multiple channel operations, picks whichever is ready
select {
case msg := <-ch1:
    fmt.Println("Received from ch1:", msg)
case msg := <-ch2:
    fmt.Println("Received from ch2:", msg)
case ch3 <- "hello":
    fmt.Println("Sent to ch3")
default:
    fmt.Println("No channel ready") // non-blocking with default
}

// Timeout pattern — don't wait longer than N seconds
select {
case result := <-resultCh:
    fmt.Println("Got result:", result)
case <-time.After(5 * time.Second):
    fmt.Println("Timed out")
}

// Done channel pattern — cancel a goroutine
func worker(done <-chan struct{}) {
    for {
        select {
        case <-done: // closed channel is always ready
            fmt.Println("Worker cancelled")
            return
        default:
            doWork()
        }
    }
}

done := make(chan struct{})
go worker(done)
close(done) // cancel — closing broadcasts to all receivers
```

4.4 sync Package — Mutexes and Atomic Operations

```
import (
    "sync"
    "sync/atomic"
)

// Mutex — mutual exclusion for shared state
type SafeCounter struct {
    mu sync.Mutex
    v  map[string]int
}
```

```
func (c *SafeCounter) Inc(key string) {
    c.mu.Lock()
    defer c.mu.Unlock() // always unlock, even if we panic
    c.v[key]++
}

// RWMutex — multiple readers OR one writer (better for read-heavy)
type Cache struct {
    mu    sync.RWMutex
    data map[string]string
}

func (c *Cache) Get(key string) (string, bool) {
    c.mu.RLock(); defer c.mu.RUnlock() // shared read lock
    v, ok := c.data[key]; return v, ok
}

func (c *Cache) Set(key, value string) {
    c.mu.Lock(); defer c.mu.Unlock() // exclusive write lock
    c.data[key] = value
}

// sync.Once — run exactly once (for singletons, lazy init)
var (
    instance *Database
    once      sync.Once
)

func GetDB() *Database {
    once.Do(func() { instance = newDatabase() })
    return instance
}

// Atomic operations — for simple counters (faster than mutex)
var requestCount int64
atomic.AddInt64(&requestCount, 1)
count := atomic.LoadInt64(&requestCount)
```

4.5 context — Cancellation and Deadlines

```
import "context"

// context propagates cancellation, deadlines, and values across goroutines

// WithCancel — manual cancellation
ctx, cancel := context.WithCancel(context.Background())
defer cancel() // always call cancel to release resources

// WithTimeout — cancel after duration
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

// WithDeadline — cancel at specific time
deadline := time.Now().Add(10 * time.Second)
ctx, cancel := context.WithDeadline(context.Background(), deadline)
defer cancel()
```

```
// Use context in functions – always first parameter, always named ctx
func fetchUser(ctx context.Context, id int) (*User, error) {
    req, _ := http.NewRequestWithContext(ctx, "GET",
        fmt.Sprintf("/api/users/%d", id), nil)
    resp, err := http.DefaultClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()
    var user User
    json.NewDecoder(resp.Body).Decode(&user)
    return &user, nil
}

// Respect context cancellation in long-running operations
func processItems(ctx context.Context, items []Item) error {
    for _, item := range items {
        select {
        case <-ctx.Done(): return ctx.Err() // cancelled or timed out
        default:
            if err := processItem(ctx, item); err != nil { return err }
        }
    }
    return nil
}
```

Phase 4 Projects



Concurrent Web Crawler

Crawl URLs concurrently using goroutines and channels. A work queue channel, a results channel, a WaitGroup to know when all crawlers are done. Respect context cancellation. Rate limit requests per domain.

Concepts: [goroutines](#) · [channels](#) · [WaitGroup](#) · [context](#) · [rate limiting](#) · [sync.Mutex](#)



Fan-Out/Fan-In Pipeline

Build a data processing pipeline: generate() → multiple parallel process() workers (fan-out) → merge() (fan-in) → output. Use done channel for clean shutdown.

Concepts: [channels](#) · [goroutines](#) · [fan-out](#) · [fan-in](#) · [select](#) · [done channel](#)

PHASE 5 Standard Library*net/http, encoding/json, io, os, and context*

5.1 Building HTTP Servers with net/http

```
package main

import (
    "encoding/json"
    "log"
    "net/http"
)

type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Email string `json:"email,omitempty"`
}

// Handler function signature: (ResponseWriter, *Request)
func usersHandler(w http.ResponseWriter, r *http.Request) {
    switch r.Method {
    case http.MethodGet:
        users := []User{{ID: 1, Name: "Alice"}, {ID: 2, Name: "Bob"}}
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(users)

    case http.MethodPost:
        var user User
        if err := json.NewDecoder(r.Body).Decode(&user); err != nil {
            http.Error(w, err.Error(), http.StatusBadRequest)
            return
        }
        user.ID = 3 // assign ID
        w.Header().Set("Content-Type", "application/json")
        w.WriteHeader(http.StatusCreated)
        json.NewEncoder(w).Encode(user)

    default:
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
    }
}

// Middleware — wraps a handler
func logMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        log.Printf("%s %s", r.Method, r.URL.Path)
        next.ServeHTTP(w, r)
    })
}

func main() {
    mux := http.NewServeMux()
}
```

```
    mux.HandleFunc("/users", usersHandler)
    mux.Handle("/static/", http.StripPrefix("/static/",
http.FileServer(http.Dir("./static"))))

    server := &http.Server{
        Addr:      ":8080",
        Handler:    logMiddleware(mux),
        ReadTimeout: 10 * time.Second,
        WriteTimeout: 10 * time.Second,
    }
    log.Fatal(server.ListenAndServe())
}
```

5.2 encoding/json — Serialisation

```
// Struct tags control JSON encoding/decoding
type Product struct {
    ID      int      `json:"id"`
    Name    string   `json:"name"`
    Price   float64  `json:"price"`
    Description string `json:"description,omitempty"` // omit if empty
    internal string  // unexported - never serialised
}

// Marshal - Go value to JSON bytes
p := Product{ID: 1, Name: "Widget", Price: 9.99}
data, err := json.Marshal(p)
// data = {"id":1,"name":"Widget","price":9.99}

// MarshalIndent - pretty print
data, err = json.MarshalIndent(p, "", " ")

// Unmarshal - JSON bytes to Go value
jsonStr := `{"id":1,"name":"Widget","price":9.99}`
var product Product
err = json.Unmarshal([]byte(jsonStr), &product)

// Streaming - better for HTTP (avoids buffering)
// Encoder/Decoder operate on io.Writer/io.Reader
json.NewEncoder(w).Encode(product) // to HTTP response
json.NewDecoder(r.Body).Decode(&product) // from HTTP request

// Dynamic JSON with map
var raw map[string]interface{}
json.Unmarshal(data, &raw)
name := raw["name"].(string) // type assertion needed

// Recommended: use json.RawMessage for partial parsing
type Envelope struct {
    Event string      `json:"event"`
    Data  json.RawMessage `json:"data"` // parse Data later based on Event
}
```

Phase 5 Projects



REST API Server

A full REST API: CRUD for a resource (e.g. books), JSON request/response, middleware stack (logging, recovery, auth), query params for filtering, proper HTTP status codes throughout.

Concepts: [net/http](#) · [encoding/json](#) · [ServeMux](#) · [middleware](#) · [http.StatusCode](#) · [context](#)

PHASE 6 Modules, Testing & Tooling

The Go toolchain — the best in class

6.1 Go Modules

```
# Initialise a module
go mod init github.com/yourname/myproject

# go.mod — the module manifest
module github.com/yourname/myproject

go 1.22

require (
    github.com/gin-gonic/gin v1.9.1
    github.com/jackc/pgx/v5 v5.5.0
)

# Common commands
go get github.com/some/package           # add dependency
go get github.com/some/package@v1.2.3    # specific version
go mod tidy                               # remove unused, add missing
go mod vendor                             # copy deps to ./vendor
go list -m all                            # list all dependencies
go build ./...                             # build all packages
go run .                                  # run the main package
go run ./cmd/server                       # run a specific command
go vet ./...                              # static analysis
go fmt ./...                              # format all code
staticcheck ./...                         # third-party linter
```

6.2 Testing

```
// math.go
package math

func Add(a, b int) int { return a + b }
func Divide(a, b float64) (float64, error) {
    if b == 0 { return 0, errors.New("division by zero") }
    return a / b, nil
}

// math_test.go — file must end in _test.go
package math

import (
    "testing"
```

```

    "errors"
)

func TestAdd(t *testing.T) {
    got := Add(2, 3)
    want := 5
    if got != want {
        t.Errorf("Add(2,3) = %d; want %d", got, want)
    }
}

// Table-driven tests — idiomatic Go
func TestDivide(t *testing.T) {
    tests := []struct {
        name      string
        a, b       float64
        want       float64
        wantErr    bool
    }{
        {"normal", 10, 2, 5, false},
        {"fractions", 10, 3, 3.333, false},
        {"div by zero", 5, 0, 0, true},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got, err := Divide(tt.a, tt.b)
            if (err != nil) != tt.wantErr {
                t.Errorf("Divide() error = %v, wantErr %v", err, tt.wantErr)
                return
            }
            if !tt.wantErr && math.Abs(got-tt.want) > 0.001 {
                t.Errorf("Divide() = %v, want %v", got, tt.want)
            }
        })
    }
}

// go test ./...           run all tests
// go test -v ./...        verbose output
// go test -run TestDivide run specific test
// go test -race ./...      run with race detector
// go test -cover ./...     show coverage
// go test -bench=. ./...   run benchmarks

// Benchmark
func BenchmarkAdd(b *testing.B) {
    for i := 0; i < b.N; i++ { Add(2, 3) }
}

```

Phase 6 Projects



Test a REST API

Write a full test suite for the API from Phase 5: unit tests for handlers using `httptest.NewRecorder`, integration tests using `httptest.NewServer`, table-driven tests, and a benchmark for the most common endpoint.

Concepts: *testing.T* · *httptest* · *table-driven tests* · *benchmarks* · *race detector* · *-cover*

PHASE 7 Advanced Go*Generics, reflection, build system, and the ecosystem*

7.1 Generics (Go 1.18+)

```
// Type parameters - T is constrained to 'comparable'
func Contains[T comparable](slice []T, item T) bool {
    for _, v := range slice { if v == item { return true } }
    return false
}
Contains([]int{1,2,3}, 2)           // true
Contains([]string{"a","b"}, "c") // false

// Custom type constraint
type Number interface { int | int32 | int64 | float32 | float64 }

func Sum[T Number](slice []T) T {
    var total T
    for _, v := range slice { total += v }
    return total
}
Sum([]int{1, 2, 3})           // 6
Sum([]float64{1.5, 2.5})     // 4.0

// Generic data structures
type Stack[T any] struct { items []T }
func (s *Stack[T]) Push(item T)    { s.items = append(s.items, item) }
func (s *Stack[T]) Pop() (T, bool) {
    if len(s.items) == 0 { var zero T; return zero, false }
    n := len(s.items) - 1
    item := s.items[n]
    s.items = s.items[:n]
    return item, true
}

s := &Stack[string]{}
s.Push("hello"); s.Push("world")
v, _ := s.Pop() // "world"
```

7.2 The Go Ecosystem

Category	Package / Tool	Purpose
Web Framework	Gin	Fast HTTP router, middleware, JSON binding
Web Framework	Echo	Minimalist high-performance router

Web Framework	Chi	Lightweight, idiomatic, uses stdlib interfaces
HTTP Client	resty	Fluent REST client with retry, auth, middleware
Database	pgx	PostgreSQL driver — direct and via database/sql
Database	sqlx	Extends database/sql with struct scanning
ORM	GORM	Full-featured ORM for PostgreSQL, MySQL, SQLite
Config	Viper	Config files, env vars, flags — all in one
CLI	Cobra	The standard library for building CLI tools (kubect! uses it)
Logging	zerolog / zap	Structured, high-performance JSON logging
Testing	testify	Assertions, mocks, test suites
Validation	validator	Struct tag-based validation

Capstone Projects



Key-Value Store

A TCP-based key-value store from scratch: RESP protocol parsing, SET/GET/DEL/EXPIRE commands, in-memory hash map with RWMutex, persistence via AOF (append-only file), concurrent client connections.

Concepts: [net.Listen](#) · [goroutines](#) · [channels](#) · [sync.RWMutex](#) · [io](#) · [encoding](#)



CLI Tool with Cobra

A useful CLI tool built with Cobra: multiple subcommands, config via Viper (YAML + env vars), progress bars, coloured output, and a test suite. Publish as a binary release on GitHub.

Concepts: [cobra](#) · [viper](#) · [os/exec](#) · [testing](#) · [goreleaser](#)

You have reached the end of the Go coursework.

Write a server, run the race detector, read the standard library. Go rewards the curious.